Locks and Deadlocks

shared vs exclusive, and why lock order matters

Watch the 3-min video

In one line: databases protect concurrent writes with shared and exclusive locks; a deadlock is a cycle of transactions each waiting for a lock the other holds. The database breaks it by aborting one, and you prevent it by always locking in the same order.

1 The problem: two transfers, opposite directions

Two transfers run at the same moment. Transfer 1 moves money from account A to B. Transfer 2 moves money from B to A. Each locks its **source** account first, then waits for the other's. Forever. That is a **deadlock**.

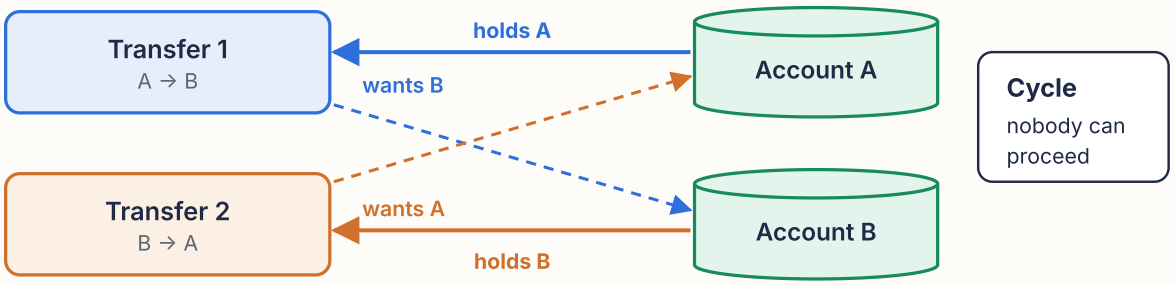


Figure 1. Transfer 1 holds A and wants B; Transfer 2 holds B and wants A. The waits form a cycle.

2 Lock types

- **Row locks** protect individual rows; **table locks** protect whole tables.
- A **shared lock** (S) lets many transactions read, and blocks writers.
- An **exclusive lock** (X) gives one transaction write access, and blocks everyone else who wants a lock.

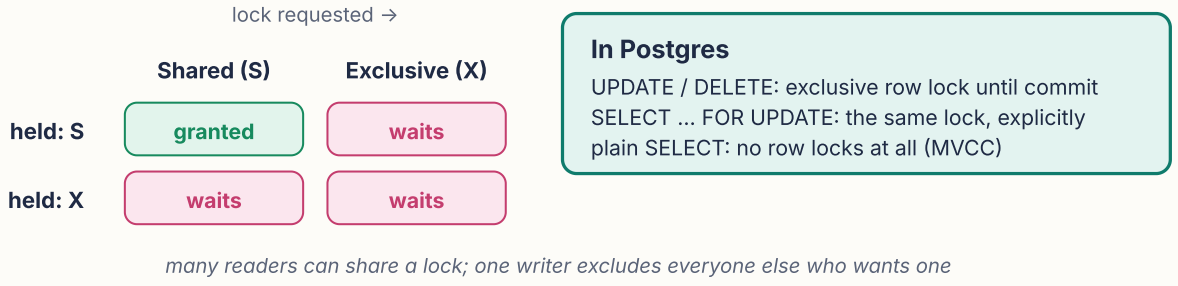


Figure 2. Lock compatibility: only shared + shared can be held together. In Postgres, plain reads skip row locks entirely thanks to MVCC.

3 Two traps: lock escalation and schema changes

Lock escalation. Some databases, like SQL Server, upgrade thousands of row locks held by one transaction into a single table lock, to save memory (SQL Server tries at about 5,000 locks on one table). That's cheaper, but it can suddenly block everyone else on the table. Postgres does not escalate.

Schema changes. An `ALTER TABLE` needs a strong table lock. If it queues behind one long query, **all traffic queues behind it**, because new queries can't jump the waiting exclusive request.



Figure 3. A harmless-looking migration stuck behind a long query takes the table offline. Set lock_timeout on migrations (e.g. 2–5 s) and retry.

4 How deadlocks are detected and prevented

A deadlock needs a **cycle** of waits. Databases detect it: Postgres checks for cycles after a transaction has waited **1 second** (`deadlock_timeout`), then aborts one transaction with `ERROR: deadlock detected`. The other continues. Your application must **catch that error and retry**.

The best prevention is **consistent lock ordering**. If every transfer locks the **lower account ID first**, then the higher one, a cycle is impossible: both transfers try to lock A first, so one simply waits for the other.

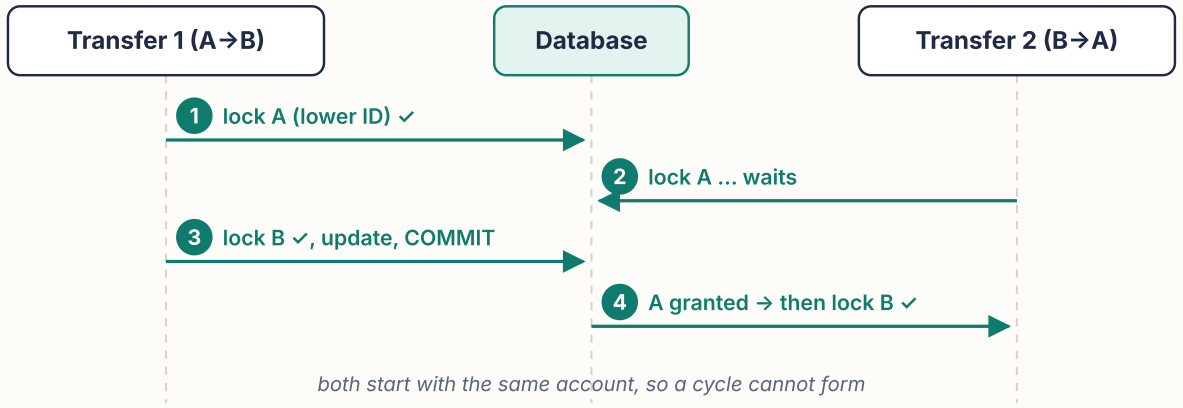


Figure 4. With a global order (lowest ID first), Transfer 2 waits at step 2 instead of grabbing B. It proceeds as soon as Transfer 1 commits.

WORKED EXAMPLE: A DEADLOCK-FREE TRANSFER

```
BEGIN;
-- lock both accounts up front, always in ID order
SELECT id, balance FROM accounts
 WHERE id IN (:from, :to)
 ORDER BY id
 FOR UPDATE;
UPDATE accounts SET balance = balance - :amt WHERE id = :from;
UPDATE accounts SET balance = balance + :amt WHERE id = :to;
COMMIT;
```

Transfers 1→2 and 2→1 both lock account 1 first. The second waits a few milliseconds, then runs. No cycle, no abort.

5 Other tactics, and debugging

- **Keep transactions short.** Never wait for user input or an external API while holding locks.
- **Lock what you need up front**, instead of adding locks as you go.

- **Set a lock timeout** (`SET lock_timeout = '2s'`) so a blocked statement fails fast instead of hanging.
- **Job queues: use SKIP LOCKED.** `SELECT … FOR UPDATE SKIP LOCKED LIMIT 1` lets each worker skip rows another worker already holds.
- **Debug it.** Postgres logs every deadlock with the queries involved. The `pg_locks` and `pg_stat_activity` views show who is blocking whom right now (`pg_blocking_pids()` helps). Then look for code paths that touch the same rows in a different order.

COMMON MISTAKES

- Locking rows in whatever order the request names them (from, to) instead of a global order.
- Treating “deadlock detected” as a bug to log and drop. It is expected under load: retry the whole transaction.
- Calling a payment API inside an open transaction that holds row locks.
- Running `ALTER TABLE` on a busy table with no `lock_timeout`.

6 Key terms

Shared lock (S): many readers; blocks writers

Exclusive lock (X): one writer; blocks all other lockers

Row / table lock: protects one row / the whole table

Deadlock: a cycle of transactions waiting on each other

Lock ordering: always acquire locks in one global order

Lock escalation: many row locks become one table lock

lock_timeout: give up waiting for a lock after N

SKIP LOCKED: skip rows already locked by others

deadlock_timeout: Postgres wait before cycle check (1 s)

7 Interview questions

Q1. How do deadlocks occur? BASIC

Two or more transactions each hold a lock the other needs, forming a cycle, like two transfers locking accounts in opposite order. The database detects the cycle and aborts one of them.

Q2. How do you prevent deadlocks? BASIC

Acquire locks in a consistent order (e.g. by account ID). Keep transactions short, lock what you need up front, use lock timeouts, and retry transactions that fail with a deadlock error.

Q3. Shared vs exclusive lock? BASIC

Shared: many holders at once, for reading; it blocks writers. Exclusive: one holder, for writing; it blocks every other lock request on that row or table.

Q4. Your transfer service logs ~20 deadlocks a day. What do you do? SCENARIO

Read the deadlock log entries to see the queries and rows. Typically two code paths lock the same accounts in different orders. Lock both accounts up front with SELECT ... ORDER BY id FOR UPDATE, and add a retry (up to ~3 attempts) on deadlock errors.

Q5. Ten workers pull jobs from one table and keep blocking each other. Fix it. SCENARIO

Use SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1 so each worker takes the next unlocked job instead of waiting on the same first row.

Q6. A 1-second migration took the site down for 10 minutes. How? SCENARIO

The ALTER TABLE waited for an exclusive lock behind a long-running query, and every new query queued behind the ALTER. Set lock_timeout on migrations so they fail fast and retry, and kill or avoid long queries first.

Q7. Postgres aborted one transaction in a deadlock. Which one, and is the other safe? TRICKY

Usually the transaction whose 1-second wait ran the cycle check; you can't choose. The other proceeds normally and commits. The aborted one lost all its work and must be retried from the start.

Q8. Do plain SELECTs in Postgres take row locks or cause deadlocks with writers? TRICKY

No. Thanks to MVCC, plain reads take no row locks and never block or are blocked by row writers. They do take a weak table lock, which is why they can block ALTER TABLE.

Next: SD-E-012 · Optimistic vs Pessimistic Locking backendroadmap.com